Report: Identifying Key Entities in Recipe Data

Business Objective:
The goal of this assignment is to train a Named Entity Recognition (NER) model using Conditional Random Fields (CRF) to extract key entities from recipe data. The model will classify words into predefined categories such as ingredients, quantities and units, enabling the creation of a structured database of recipes and ingredients that can be used to power advanced features in recipe management systems, dietary tracking apps, or e-commerce platforms.

1. Import libraries

1.1. Import necessary libraries

- Imported **JSON** for data loading.
- **Pandas** and **Numpy** from data manipulation and analysis
- **Matplotlib.pyplot** and **Seaborn** for visualisation
- **Sklearn** for model building and evaluation
- **Sklearn_crfsuite** for *CRF (Conditional Random Fields) implementation for sequence modeling*
- **Spacy** for english vocabulary
- **Re** (regular expressions) for text preprocessing
- **Counter**, **random** for computation

2. Data Ingestion and Preparation

2.1. Recipe Data from Dataframe and prepare the data for analysis

- The recipe data was loaded from the "**ingredient_and_quantity.json**" file into a pandas Dataframe named df using the custom load_json_dataframe function.
- Initially, the Dataframe df contained **285** rows and **2** columns (input and pos).
- The df.info() output confirmed that both columns were of object type and contained no null values.

2.1.1. Create input_tokens and pos_tokens columns by splitting the input and pos from the dataframe

- The "input" and "pos" columns were split into "**input_tokens**" and "**pos_tokens**" respectively.

2.1.2. Provide the length for input_tokens and pos_tokens and validate their length

- New columns "**input_length**" and "**pos_length**" were created to store the number of tokens for each.

2.1.3. Define a unique_labels function and validate the labels in pos_tokens

- Unique Pos labels: **{'ingredient', 'unit', 'quantity'}**

2.1.4. Provide the insights seen in the recipe data after validation

- **Index [17, 27, 79, 164, 207]** contains inconsistent fraction formats, ambiguous ingredient naming, duplicate terms, and missing or misordered units, and therefore require cleaning and formatting before modelling

2.1.5. Drop the rows that have invalid data provided in previous cell

- It was identified that *5 rows had inconsistent lengths between "input_tokens" and "pos_tokens".
- These rows were considered invalid for model training and were subsequently dropped from the DataFrame to ensure data integrity and alignment between tokens and their corresponding labels.

2.1.6. Update the input_length & pos_length in dataframe

- After dropping these rows, the "input_length" & "pos_length" lengths were re-calculated and **updated**.

2.1.7. Validate the input_length and pos_length by checking unequal rows

- After dropping these rows, the lengths were re-validated, confirming no inconsistencies remained.

3. Train Validation Split

3.1. Perform train and validation split ratio

3.1.1. Split the dataset into train_df and val_df into 70:30 ratio

- 70% of the data was allocated for training and 30% for validation

3.1.2. Print the first five rows of train_df and val_df

Train_df:

	input	pos	input_tokens	pos_tokens	input_length	pos_length	length_equal
179	250 grams Okra Oil 1 Onion finely chopped Tomato Grated teaspoon Ginger 2 Garlic Finely 1/2 Cumin seeds 1/4 Teaspoon asafoetida cup cottage cheese pinched coriander powder mango red chilli turmeric	quantity unit ingredient ingredient quantity ingredient ingredient ingredient ingredient ingredient unit ingredient quantity ingredient ingredient ingredient quantity unit ingredient ingredient ingredient quantity unit ingredient unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient	[250, grams, Okra, Oil, 1, Onion, finely, chopped, Tomato, Grated, teaspoon, Ginger, 2, Garlic, Finely, 1/2, Cumin, seeds, 1/4, Teaspoon, asafoetida, cup, cottage, cheese, pinched, coriander, powder, mango, red, chilli, turmeric]	[quantity, unit, ingredient, ingredient, quantity, ingredient, ingredient, ingredient, ingredient, ingredient, unit, ingredient, quantity, ingredient, ingredient, ingredient, quantity, ingredient, ingredient, quantity, unit, ingredient, unit, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient, ingredient]	31	31	True
57	200 grams Paneer Homemade Cottage Cheese 2 Potato Aloo Bay leaf tej patta Dry Red Chilli 1 tablespoon Panch Phoran Masala roasted and powdered Tomato big sized teaspoon Turmeric powder Haldi Cumin seeds Jeera Ginger grated Salt 1/2 Sugar Sunflower Oil	quantity unit ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient ingredient

4. Exploratory Recipe Data Analysis on Training Dataset

4.1. Flatten the lists for input_tokens & pos_tokens

- **Flatten_list** function defined for flattening the structure for input_tokens and pos_tokens.

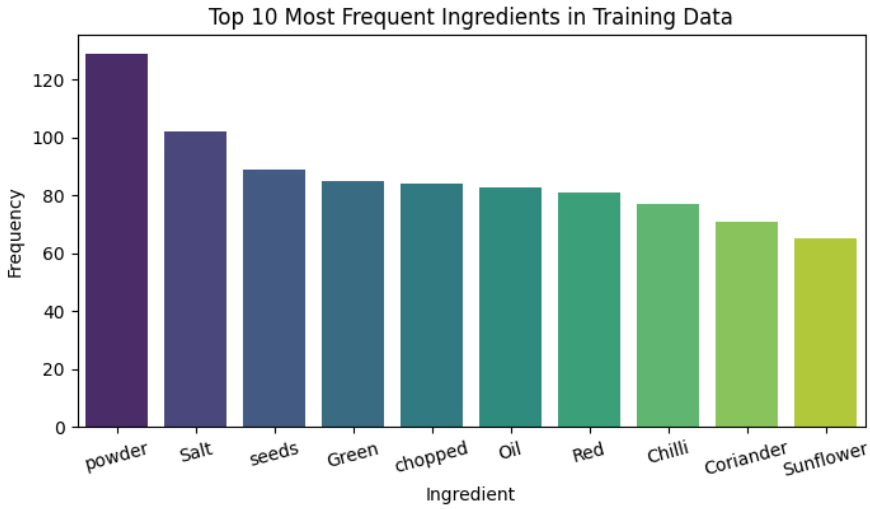
4.2. Extract and validate the tokens after using the flattening technique

```
--- Training Dataset Token Validation ---
Length of flattened input_tokens: 7114
Length of flattened pos_tokens: 7114

First 10 input_tokens for Training:
['250', 'grams', 'Okra', 'Oil', '1', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated']

First 10 pos_tokens for Training:
['quantity', 'unit', 'ingredient', 'ingredient', 'quantity', 'ingredient', 'ingredient', 'ingredient', 'ingredient', 'ingredient']

--- Validation Dataset Token Validation ---
Length of flattened input_tokens: 2876
Length of flattened pos_tokens: 2876

First 10 input_tokens for Validation:
['1', 'cup', 'Ada', '2', 'liter', 'Milk', '3/4', 'Sugar', 'tablespoon', 'Ghee']

First 10 pos_tokens for Validation:
['quantity', 'unit', 'ingredient', 'quantity', 'unit', 'ingredient', 'quantity', 'ingredient', 'unit', 'ingredient']
```

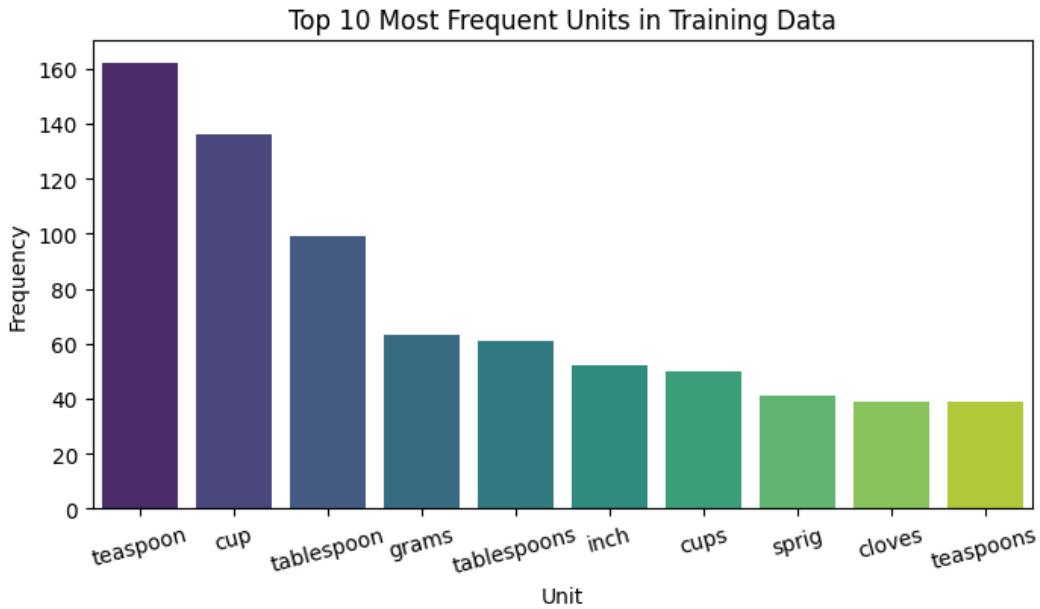
4.3. Categorise tokens into labels (unit, ingredient, quantity)

- Sample categorized tokens:
  - **Ingredients**: ['Okra', 'Oil', 'Onion', 'finely', 'chopped', 'Tomato', 'Grated', 'Ginger', 'Garlic', 'Finely']
  - **Units**: ['grams', 'teaspoon', 'Teaspoon', 'cup', 'grams', 'tablespoon', 'teaspoon', 'grams', 'teaspoon', 'sprig']
  - **Quantities**: ['250', '1', '2', '1/2', '1/4', '200', '2', '1', '1/2', '500']

4.4. Top 10 Most Frequent Ingredients



4.5. Plot Top 10 most frequent units

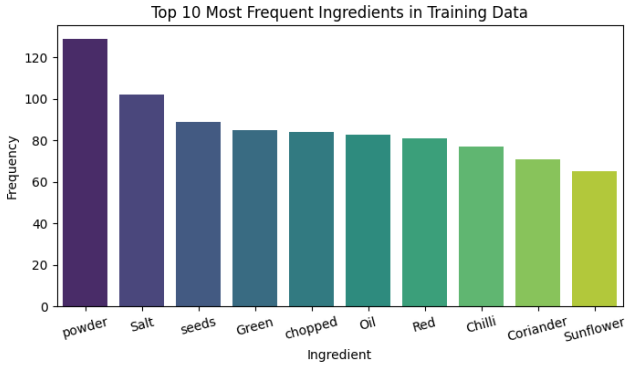


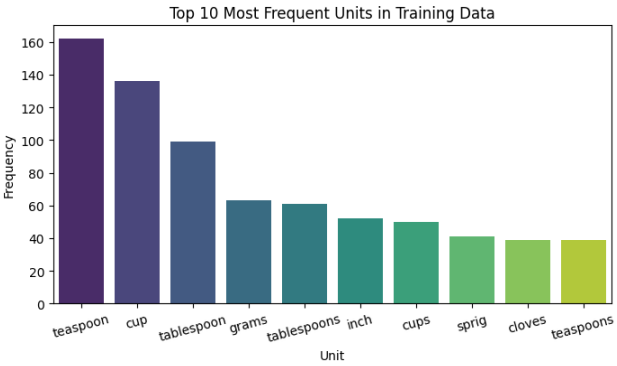
4.6. Perform EDA analysis

- The ingredient distribution shows **powder** and **salt** as the most frequent terms, indicating heavy use of powdered spices and basic seasonings.
- Common terms such as **chopped**, **green** and **red** suggest frequent use of preparation descriptors rather than pure ingredient names.
- The units plot is dominated by **teaspoon** and **cup**, reflecting standard household measurements.
- Variations like **teaspoon/teaspoons**, **cup/cups**, and **tablespoon/tablespoons** indicate inconsistent unit formatting

5. Exploratory Recipe Data Analysis on Validation Dataset

5.1. Execute EDA on Validation Dataset with insights





- Similar to the training dataset, **powder** and **salt** remain the most frequent ingredients in the validation set, indicating consistency in core ingredient usage across splits.
- Ingredients such as **oil, red, seeds, chilli, chopped, and green** also appear prominently, showing that the validation data follows the same cooking patterns and vocabulary as the training data.
- Minor differences like the presence of **leaves** in validation and **sunflower** in training reflect natural variation rather than data drift.
- The unit distribution closely mirrors the training dataset, with **teaspoon** and **cup** being the dominant measurement units.
- As observed in training, plural and singular forms (**cup/cups**, **teaspoon/teaspoons**, **tablespoon/tablespoons**) coexist, confirming consistent unit-format inconsistencies across datasets.
- Overall, the validation dataset aligns well with the training dataset in both ingredient and unit distributions, suggesting a well-balanced split and supporting reliable model generalization

6. Feature Extraction For CRF Model

6.1. Define a feature functions to take each token from recipe

To enhance feature extraction, specific sets of keywords and a regular expression pattern were defined:

- **unit_keywords**: This set was populated with words identified as common measurement units from the `unit_list` generated by `categorize_tokens`.
- **quantity_keywords**: This set included common terms indicating quantities, derived from the `quantity_list` generated by `categorize_tokens`.
- **quantity_pattern**: A robust regular expression was defined (`re.compile`) to identify various numeric forms representing quantities, including:
    - Fractions (e.g., "1/2", "1 1/2")
    - Decimals (e.g., "1.5")
    - Whole numbers (e.g., "1")
    - Unicode fractions (e.g., "¼½¾")

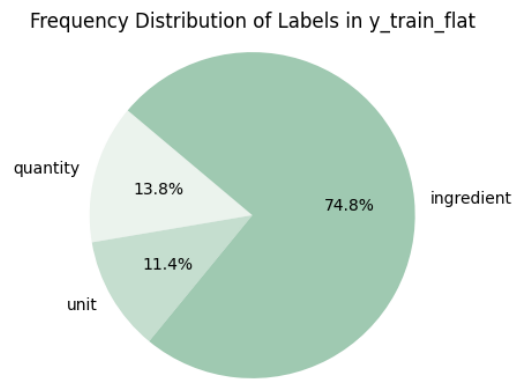
These definitions were crucial for the `**word2features**` function to accurately detect and flag quantities and units.

6.2. Preparation of Recipe level features

- The word2features function extracts various linguistic and contextual features for a single token.
- Finally, **sent2features** returns a list of feature dictionaries, where each dictionary corresponds to a token in the original sentence and contains all its extracted features.
- This process transforms raw tokenized sentences into a format suitable for training the Conditional Random Field (CRF) model.

6.3. Convert X_train, X_val, y_train and y_val into train and validation feature sets and labels

- It takes the tokenized sentences from X_train and X_val and applies the sent2features function to each one.
- Y_train_label and y_val_label which likely hold the lists of Part-of-Speech (POS) labels for the training and validation datasets, and explicitly converts them into standard Python lists.
- Number of X_train_features and y_train_lables is **196**
- Number of X_val_features and y_val_lables is **84**



6.4. Applying weights to feature sets

- Label: quantity, Count: 980, Frequency: 0.1378
- Label: unit, Count: 811, Frequency: 0.1140
- Label: ingredient, Count: 5323, Frequency: 0.7482

7. Model Building and Training

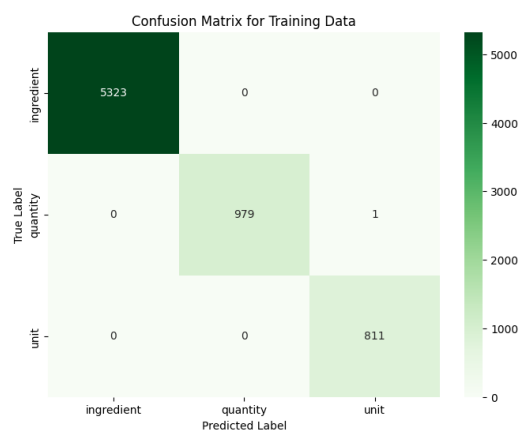
7.1. Initialise the CRF model and train it

Initialise CRF model with tunned hyperparameter.

- **algorithm='lbfgs'**: Optimisation algorithm used for training. lbfgs (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) is a quasi-Newton optimisation method.
- **c1=0.5** : L1 regularisation term to control sparsity in feature weights. Helps in feature selection.
- **c2=1.0** : L2 regularisation term to prevent overfitting by penalising large weights.
- **max_iterations=100** : Maximum number of iterations for model training. Higher values allow more convergence but increase computation time.
- **all_possible_transitions=True** : Ensures that all possible state transitions are considered in training, making the model more robust.

7.2. Evaluation of Training Dataset using CRF model

```
Training Classification Report:
              precision    recall  f1-score   support

  ingredient     1.0000    1.0000    1.0000      5323
    quantity     1.0000    0.9959    0.9980       980
        unit     0.9951    1.0000    0.9975       811

    accuracy                         0.9994      7114
   macro avg     0.9984    0.9986    0.9985      7114
weighted avg     0.9994    0.9994    0.9994      7114
```



- The confusion matrix showed almost perfect classification

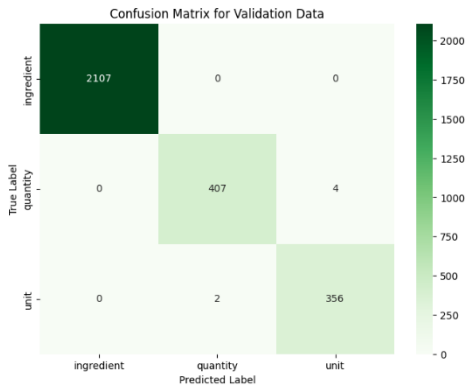
7.3. Save the CRF model

- dump the model using joblib as crf_model.pkl

8. Prediction and Model Evaluation

8.1. Predict and Evaluate the CRF model on validation set

	precision	recall	f1-score	support
ingredient	1.0000	1.0000	1.0000	2107
quantity	0.9951	0.9903	0.9927	411
unit	0.9889	0.9944	0.9916	358
accuracy			0.9979	2876
macro avg	0.9947	0.9949	0.9948	2876
weighted avg	0.9979	0.9979	0.9979	2876



- The confusion matrix indicated strong performance with very few misclassifications.

9. Error Analysis on Validation Data

9.1. Investigate misclassified samples in validation dataset

- An in-depth error analysis was conducted on the validation set, which showed an overall accuracy of **99.79%.**
- Label-Specific Errors:
  - **Quantity**: 4 misclassifications out of 411 instances (**Error Rate: 0.004082**)
  - **Unit**: 2 misclassifications out of 358 instances (**Error Rate: 0.002466**)

9.2. Provide insights from the validation dataset

- The validation results show strong performance, with very low error rates for both labels (unit ≈ 0.24%, quantity ≈ 0.4%). The few errors observed are systematic rather than random.
- **Key insights from the detailed misclassifications:**
  - **Context-driven confusion:** Tokens such as “to” and “a” are sometimes misclassified as quantity when they appear near numbers (e.g., “10 to 12”, “Haldi a pinch”), due to their proximity to numeric expressions.
  - **Ambiguity in culinary measures:** Words like “is”, “for”, “pinch” and “cloves” are occasionally predicted as unit instead of quantity. These act as implicit quantities in recipe text, leading to overlap between the two classes.
  - **Neighbouring ingredient influence:** Some errors occur when adjacent tokens are ingredient names (e.g., “Haldi pinch”, “cloves Garlic”), showing that local context can bias predictions.

- **Overall conclusion**:

  With only 6 misclassified tokens in the validation set, the model generalises well. Remaining errors stem from linguistic edge cases, suggesting that phrase-level handling of numeric ranges and implicit quantities could further improve accuracy.

10. Conclusion

- **Model Strengths**:
  - The CRF model demonstrates strong performance in extracting ingredients, quantities, and units from recipe text, achieving high accuracy on both training and validation sets. Its ability to leverage sequence-based features and contextual information is evident in its robust performance.
  - with very low error rates. Most misclassifications were due to **linguistic ambiguities** in recipe text, such as implicit quantities (e.g., *pinch*, *cloves*) and connector words near numeric ranges.
- **Model Weaknesses & Areas for Improvement**:
  - While overall performance is excellent, the minor misclassifications highlight linguistic edge cases and potential ambiguities.
  - The model occasionally confuses words that can function as implicit quantities or units, or where their classification is highly context-dependent.
- Overall, the results show that CRF is a **reliable and interpretable approach** for entity extraction in recipe data, with scope for improvement through phrase-level or hybrid methods.